CPSC 481
Artificial Intelligence

Dr. Anand Panangadan
apanangadan@fullerton.edu

What we will cover today

- Coding search algorithms in Python
  - https://github.com/aimacode/aima-python

Sliding tile puzzle

solving

https://www.helpfulgames.com/subjects/brain-training/sliding-puzzle.html

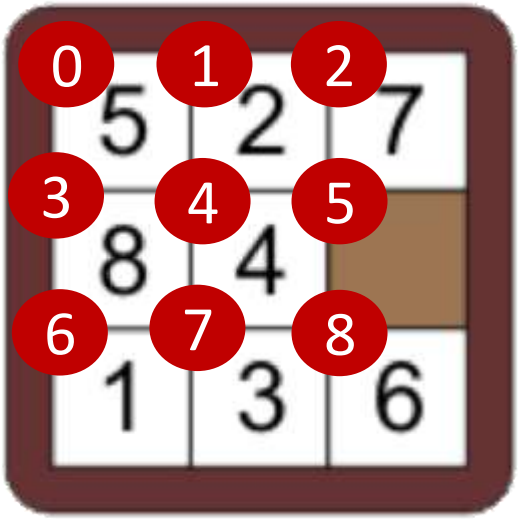
State space of the sliding tile puzzle

- State:
  - Sequence of tiles
  - Number of tiles
  - Dimensions of the board
  - Information on the board position

State space of the sliding tile puzzle

- State:
  - A 9-element tuple/array
  - Blank represented by 0
- For example:

  (5,2,7,8,4,0,1,3,6)



- Goal state?

1	4	3
7		6
5	8	2

State space of the sliding tile puzzle

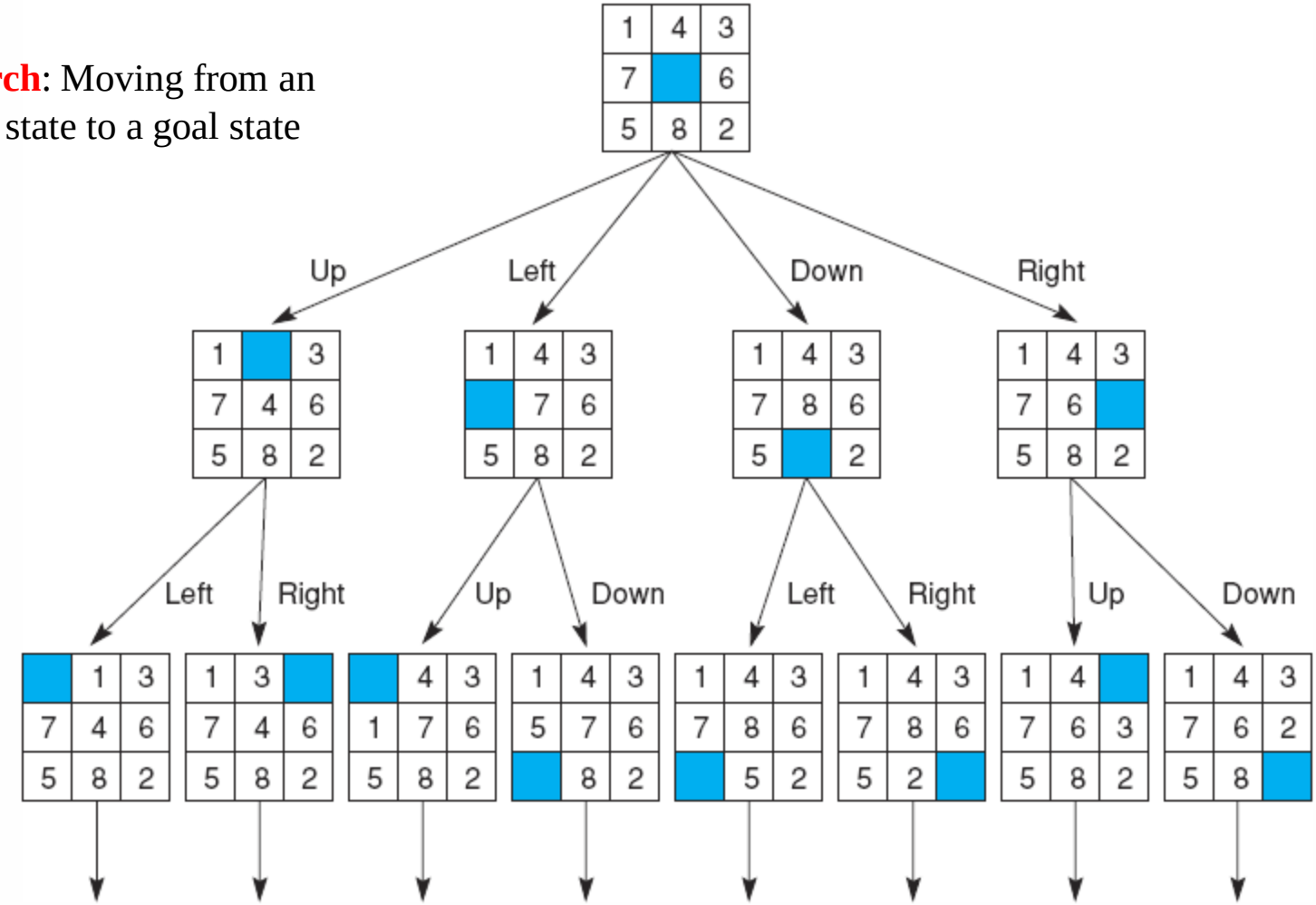
- operators: one for sliding each square in each of four directions
  - Better, one for moving the blank square in each of four directions

```
possible_actions = ['UP', 'DOWN', 'LEFT', 'RIGHT'] #
Python list
```

- What are the valid actions in a given state?

A State Space Graph for the 8-puzzle Generated by "move blank" Operations

***Search**: Moving from an initial state to a goal state



What are the valid actions in a given state?

→ [UP, DOWN, LEFT]

(5,2,7,8,4,0,1,3,6) →

(1,8,5,3,4,2,6,0,7) →

(5,2,7,8,0,4,1,3,6) →

(1,8,5,3,4,2,6,7,0) →

State space of the sliding tile puzzle

- What are the valid actions in a given state?

```
def find_blank_square(self, state):
     return state.index(0)
```

```
#Return the actions that can be executed in the given state.
def actions(self, state):

    possible_actions = ['UP', 'DOWN', 'LEFT', 'RIGHT']
    index_blank_square = self.find_blank_square(state)

    if index_blank_square == 0:
        return ['DOWN','RIGHT']
    if index_blank_square == 1:
        # ??
```

State space of the sliding tile puzzle

- What are the valid actions in a given state?

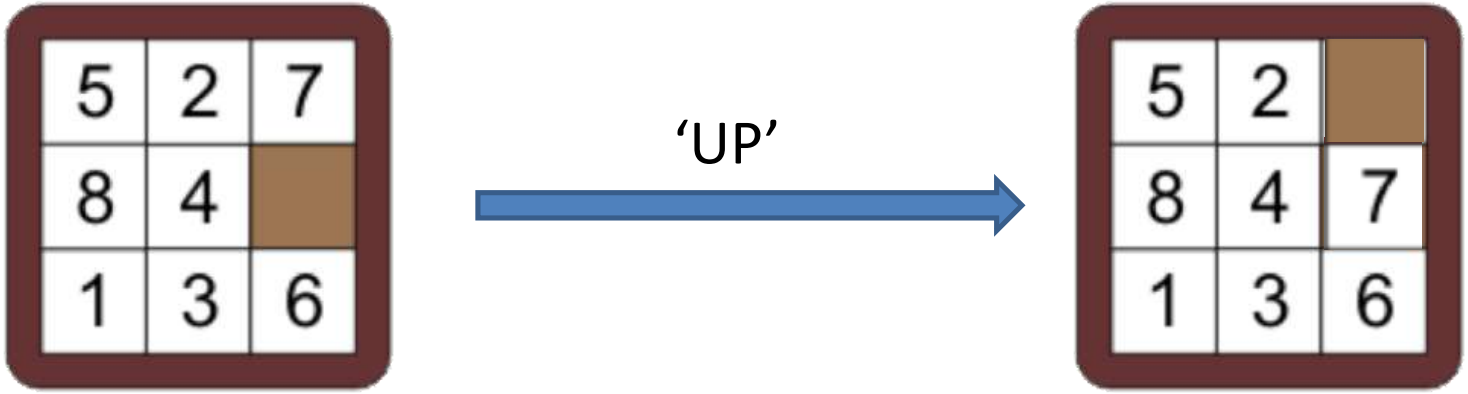
```
#Return the actions that can be executed in the given state.
def actions(self, state):

    possible_actions = ['UP', 'DOWN', 'LEFT', 'RIGHT']
    index_blank_square = self.find_blank_square(state)

    if index_blank_square % 3 == 0:
        possible_actions.remove('LEFT')
    if index_blank_square < 3:
        possible_actions.remove('UP')
    if index_blank_square % 3 == 2:
        possible_actions.remove('RIGHT')
    if index_blank_square > 5:
        possible_actions.remove('DOWN')

    return possible_actions
```

What is the *result* state after an action?



(5,2,7,8,4,0,1,3,6), 'UP' →

(5,2,7,8,4,0,1,3,6), 'LEFT' →

(5,2,7,8,0,4,1,3,6), 'DOWN' →

(1,8,5,3,4,2,6,7,0) , 'UP' →

What is the *result* state after an action?

```
def result(self, state, action):
  """ Given state and action,
      return a new state that is the result of the action.
      Action is assumed to be a valid action in the state """

  # blank is the index of the blank square
  blank = self.find_blank_square(state)
  new_state = list(state)

  delta = {'UP': -3, 'DOWN': 3, 'LEFT': -1, 'RIGHT': 1}
  neighbor = blank + delta[action]
  new_state[blank], new_state[neighbor] = new_state[neighbor], new_state[blank]

  return tuple(new_state)
```

class Problem

- A search problem has
  - Initial state
  - Goal state(s)
  - actions(state)
    - Method that returns valid actions in the given state
  - result(state, action)
    - Method that returns next state given current state, action
  - goal_test(state)
    - Is current state a goal?

class Node

```
class Node:
    #A node in a search tree.
    def __init__(self, state, parent=None, action=None, path_cost=0):
        #Create a search tree Node, derived from a parent by an action
        self.state = state
        self.parent = parent
        self.action = action


def path(self):
    #Return a list of nodes forming the path from the root to this node
    node, path_back = self, []
    while node:
        path_back.append(node)
        node = node.parent
    return list(reversed(path_back))
```

Class work

- Set up a python environment (if you haven't)
  - Get started using Python on Windows for beginners
    - https://docs.microsoft.com/en-us/windows/python/beginners
  - Other environments
    - https://www.python.org/downloads/
- Install numpy package
- Download search.py, utils.py from
  - https://github.com/aimacode/aima-python

Create & execute “mytest.py”

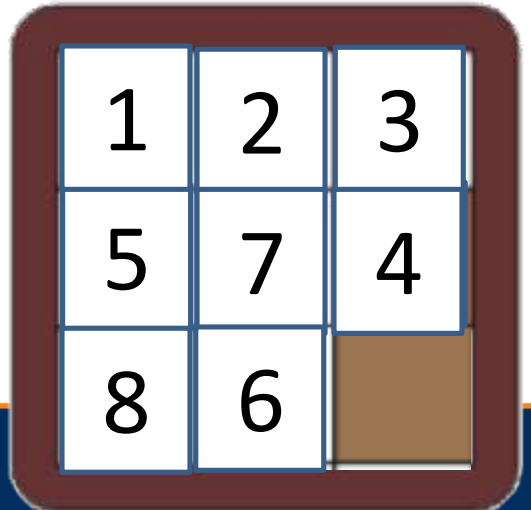
```
from search import *

eight_puzzle = EightPuzzle((1, 2, 3, 4, 5, 7, 8, 6, 0))

if __name__ == '__main__':

    print(eight_puzzle.find_blank_square((6, 3, 5, 1, 8, 4, 2, 0, 7)))
    print(eight_puzzle.actions((0, 1, 2, 3, 4, 5, 6, 7, 8))) # DOWN, RIGHT
    #print(eight_puzzle.result((0, 1, 2, 3, 4, 5, 6, 7, 8), 'DOWN'))
    #print(eight_puzzle.goal_test((1, 2, 3, 4, 5, 6, 7, 8, 0))) #true
    #print(eight_puzzle.goal_test((4, 8, 1, 6, 0, 2, 3, 5, 7))) #false
    #print(eight_puzzle.check_solvability((0, 1, 2, 3, 4, 5, 6, 7, 8)))
    #print(breadth_first_graph_search(eight_puzzle).solution())
    #print(depth_first_graph_search(eight_puzzle).solution())
```

Solve the following puzzle:



References

- AI textbook algorithms in Python
  - https://github.com/aimacode/aima-python
- Get started using Python on Windows for beginners
  - https://docs.microsoft.com/en-us/windows/python/beginners